claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\agent-a77c136ef76cc9f16.jsonl`
- SHA-256: `2542c4092f2b29659d1609b4b0cda98e707352ea6ba89d04b9355a6f472e0cac`
- Source modified: `2026-06-21T17:24:35+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `subagents`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T17:21:02.137Z)

Adversarially review PR P1a (PACKAGING_PLAN.md) in the git worktree C:/Users/avidu/Projects/bhi-wt-p1a. It adds 4 read-only engine endpoints the operator console / first-run wizard need.

Read in that worktree:
- backend/main.py: the new `/api/version`, `/api/capabilities`, `/api/videos`, `/api/reels`, `/api/reels/{filename}` handlers + `API_VERSION` + `_engine_version` + `_reels_dir` (search "API_VERSION", "capabilities", "list_reels", "download_reel").
- backend/infrastructure/database.py: the new `list_videos`.
- tests/test_api_endpoints.py: the new P1 tests.

Already verified: full suite 1284 passed/7 skipped, ruff+mypy clean, coverage 85.06%.

Adversarially check, with file:line evidence:
1. SECRET LEAK: /api/capabilities reports gemini_key_present (bool) and wasb_weights_present (bool) ? confirm it NEVER serializes the key value or a full secret path. Does it report ffmpeg_path / ffprobe_path (filesystem paths) ? is that an acceptable disclosure for an unauthenticated localhost endpoint, consistent with the existing /api/config (now redacted) and /api/videos (filepaths)?
2. /api/reels PATH SAFETY: download_reel uses safe_output_filename + resolve_under_root against _reels_dir(). Can a crafted {filename} escape output_dir (traversal, absolute, symlink, the .mp4/.mov ext gate)? Does the {filename} route capture path separators (FastAPI single-segment) ? is traversal even reachable? Is os.scandir handle closed (the `with entries:`)? Any TOCTOU?
3. /api/reels HONESTY: it's a GLOBAL flat list (no video_id association) because reels are stored flat by /api/compile ? confirm that's accurate against the compile handler, and that the docstring doesn't overclaim. Does listing top-level only (skipping subdirs) correctly exclude the per-video scratch dirs + the SQLite DB + the .lock file?
4. /api/videos: list_videos SQL is parameterized (no injection via limit)? Order-by rowid sane? Does it leak anything beyond what get_video already exposes?
5. /api/capabilities ROBUSTNESS: "never raises" ? if global_config is None, or probe_cuda_available throws, or segmenter_registry is empty, does it still 200? Is probe_cuda_available genuinely cheap (no torch import)?
6. /api/version: _engine_version fallback to "unknown" when the dist isn't installed (editable/source run) ? correct? API_VERSION contract comment accurate?

Return ONLY: findings as "[SEVERITY blocker|major|minor|nit] <file:line> ? <issue> ? <fix>", then a one-line verdict. Be precise; don't invent issues. If clean, say so.

2. assistant (2026-06-21T17:21:04.954Z)

I'll review PR P1a in the specified worktree. Let me start by reading the relevant files.

3. user (2026-06-21T17:21:06.366Z)

c039b90 feat(pkg): P0e ? single-instance lock so a 2nd process can't corrupt in-flight job state (default-OFF) (#298)

```
e8bad40 feat(pkg): P0d ? declare scipy (undeclared serving dep) + license-scan & built-wheel CI
lanes (#297)
3895963 feat(pkg): P0c ? config/profile-driven bind address + DNS-rebinding Host-header guard
(default-OFF) (#296)
fc78fce feat(pkg): P0b ? redact secret-shaped fields from the unauthenticated GET /api/config
(#295)
0c0e064 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed
engine (default-OFF) (#294)
---STATUS---
On branch feat/packaging-p1a-endpoints
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/infrastructure/database.py
        modified:   backend/main.py
        modified:   tests/test_api_endpoints.py

no changes added to commit (use "git add" and/or "git commit -a")
```

4. assistant (2026-06-21T17:21:09.024Z)

Let me read all the relevant files in the worktree.

5. user (2026-06-21T17:21:10.958Z)

```
 .gitattributes                                  |  10 +
 .github/workflows/ci.yml                        |  84 ++
 .gitignore                                      | Bin 924 -> 1117 bytes
 CLAUDE.md                                       |  19 +-
 backend/api/__init__.py                         |   1 +
 backend/api/auth.py                             | 129 ++++
 backend/api/job_worker.py                       | 184 +++++
 backend/api/uploads.py                          | 155 ++++
 backend/billing.py                              |  60 ++
 backend/compute/__init__.py                     |  61 ++
 backend/compute/base.py                         |  53 ++
 backend/compute/cloud_run.py                    | 155 ++++
 backend/config/__init__.py                      |  26 +-
 backend/config/loader.py                        | 121 ++-
 backend/config/models.py                        | 173 ++++-
 backend/config/presets.py                       | 132 ++++
 backend/config/startup.py                       | 279 +++++++
 backend/eval/ablation.py                        | 162 +---
 backend/eval/ablation_pdf.py                    | 141 ++++
 backend/eval/experiment.py                      |  82 ++
 backend/eval/fusion_audio.py                    |  43 +-
 backend/eval/fusion_compare.py                  |  39 +-
 backend/eval/golden_fixtures.py                 | 848 ++++++++++++++++++++++
 backend/eval/golden_real_fixtures.py            | 343 +++++++++
 backend/eval/rally_seg_eval.py                  | 415 +++++++++-
 backend/eval/tolerance_metrics.py               | 277 +++++++
 backend/eval/windowing.py                       |  41 +-
 backend/infrastructure/database.py              | 735 +++++++++++++-------
 backend/main.py                                 | 687 +++++++++++-------
 backend/pipeline/detectors/court_gate.py        |  99 +++
 backend/pipeline/detectors/device.py            |  56 ++
 backend/pipeline/detectors/native_wasb_runner.py | 263 +++++++
 backend/pipeline/detectors/stub_runner.py       | 157 ++++
 backend/pipeline/detectors/tracknet_runner.py   | 196 ++++-
 backend/pipeline/detectors/wasb_infer.py        |  43 +-
 backend/pipeline/segmenters/_keyhint.py         |  13 +
 backend/pipeline/segmenters/fusion_hybrid.py    |  29 +-
 backend/pipeline/segmenters/gemini.py           | 554 ++++++++------
```

```
backend/pipeline/segmenters/motion.py              |  30 +-
backend/pipeline/segmenters/trajectory_hybrid.py   |  68 +-
backend/pipeline/segmenters/wasb_hybrid.py         |   8 +
backend/pipeline/segmenters/yolo_hybrid.py         | 161 ++--
backend/pipeline/stitcher/compiler.py              | 224 +++---
backend/providers/gemini.py                        |   4 +
backend/results.py                                 |  20 +-
backend/storage/__init__.py                        | 126 +++
backend/storage/base.py                            |  49 ++
backend/storage/gcs.py                             |  63 ++
backend/storage/gdrive.py                          | 135 ++++
backend/storage/local.py                           |  69 ++
backend/storage/ref.py                             |  98 +++
backend/storage/s3.py                              |  78 ++
backend/utils/app_home.py                          | 116 +++
backend/utils/paths.py                             |  18 +-
backend/utils/redact.py                            |  51 ++
backend/utils/single_instance.py                   |  76 ++
backend/utils/workspace.py                         | 114 +++
backend/worker/__init__.py                         |   7 +
backend/worker/__main__.py                         | 517 ++++++++++++
deploy/cloudrun/Dockerfile                         |  61 ++
deploy/cloudrun/README.md                          |  61 ++
deploy/digitalocean/README.md                      |  75 ++
deploy/digitalocean/bootstrap.sh                   |  62 ++
deploy/digitalocean/gpu_setup.sh                   |  69 ++
deploy/digitalocean/mount_scratch.sh               |  46 ++
deploy/gcp/README.md                               | 257 +++++++
deploy/gcp/create_gpu_vm.sh                        |  58 ++
deploy/gcp/ops-agent-startup.sh                    |  20 +
deploy/gcp/provision.sh                            |  71 ++
deploy/gcp/teardown.sh                             |  47 ++
docs/BACKLOG.md                                    |   2 +-
docs/CLOUD_GPU_DRYRUN_GUIDE.md                     | 203 +++++
docs/CODE_MAP.md                                   |  15 +-
docs/COMMERCIALIZATION.md                          |   2 +-
docs/COMPUTE_DECOUPLED_SERVING/README.md           | 170 +++++
docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md |  83 ++
docs/COMPUTE_DECOUPLED_SERVING/architecture.svg    | 130 ++++
docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md   |  35 +
.../runlogs/RUNLOG_truly-local_TEMPLATE.md         |  49 ++
docs/DESKTOP_APP_PLAN.md                           | 164 ++++
docs/DOC_STATUS.md                                 |  92 +++
docs/EXPERIMENT_HARNESS.md                         |   2 +-
docs/GOLDEN_REGRESSION_FIXTURES.md                 | 216 ++++++
docs/HOSTING_PLAN.md                               |   4 +
docs/HOW_RALLY_DETECTION_WORKS.md                  |   2 +-
docs/I18N_PLAN.md                                  |   9 +-
docs/MULTI_SIGNAL_FUSION_PLAN.md                   |  24 +-
docs/NEXT_STEPS.md                                 |  58 +-
docs/PER_VIDEO_WORKSPACE.md                        | 112 +++
docs/PLATFORM_ARCHITECTURE.md                      |   4 +-
docs/PROJECT_DOC_TEMPLATE.md                       |  74 ++
docs/PROMINENT_COURT_DETECTION.md                  | 126 +++
docs/QUALITY_ITERATIONS.md                         | 378 ++++++++-
docs/R1_MILESTONE_LAUNCH_REPORT.md                 |  89 +++
docs/R2_EVAL_METRIC_DESIGN.md                      | 195 +++++
docs/RALLY_DETECTION_GAP_CLOSURE.md                | 104 +++
docs/RALLY_DETECTION_QUALITY_REPORT.md             | 478 ++++++++++++
docs/RALLY_QUALITY_RESEARCH.md                     |  70 +-
docs/README.md                                     |  27 +-
docs/REAL_FOOTAGE_VALIDATION_REPORT.md             | 119 +++
docs/ROADMAP.md                                    |   4 +-
docs/SETUP_NEW_MACHINE.md                          | 312 ++++++++
docs/UI_PROPOSAL.md                                |   6 +
```

```
...R_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md} |  12 +-
docs/archives/decisions/DECISIONS.md               | 205 +++++
.../field-pmf}/FIELD_VISIT_ANSWER_SHEET.md         |   0
.../field-pmf}/FIELD_VISIT_PLAYBOOK.md             |   0
docs/{ => archives/field-pmf}/PMF_FIELD_SIGNALS.md |   0
.../field-pmf}/PMF_MARKET_RESEARCH.md              |   0
.../field-pmf}/PRODUCT_FIELD_ASSOCIATE_FLYER.md    |   0
docs/archives/field-pmf/README.md                  |  15 +
.../DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md | 137 ++++
.../02-COMPUTE-MAP-AND-CONTRACT.md                 |  78 ++
.../DECOUPLED_COMPUTE/03-PLATFORM-OPTIONS.md       |  94 +++
.../DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md    | 161 ++++
.../DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md       | 105 +++
.../06-RISKS-AND-OPEN-QUESTIONS.md                 |  85 +++
.../DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md    | 168 ++++
.../DEPLOY_REPORT_GCP_2026-06-20.md                | 110 +++
.../past_projects/DECOUPLED_COMPUTE/HANDOFF.md     | 108 +++
.../past_projects/DECOUPLED_COMPUTE/README.md      | 126 +++
docs/archives/past_projects/README.md              |   3 +-
.../2026-06-18-lovo-resweep-n13/README.md          |  75 ++
.../2026-06-18-lovo-resweep-n13/lovo_resweep.py    | 111 +++
docs/archives/quality-iterations/README.md         |   1 +
.../roora-vendor}/GOLDEN_SET_VENDORING.md          |   0
docs/archives/roora-vendor/README.md               |  14 +
.../roora-vendor}/ROORA_LABELING_GUIDELINES.md     |   0
docs/{ => data_pipeline}/GOLDEN_DATA_SHARING.md    |   2 +-
.../GOLDEN_SET_IMPLEMENTATION.md                   |   2 +-
.../GOLDEN_SET_PHASE1_VIDEOS.md                    |   2 +-
docs/data_pipeline/GOLDEN_VIDEOS.md                |  60 ++
docs/data_pipeline/README.md                       |  16 +
.../VIDEO_RECORDING_GUIDELINES.md                  |  21 +
eval_baselines/fixtures/README.md                  |  70 ++
.../fixtures/clean_single_rally/expected.json      |  39 +
.../fixtures/clean_single_rally/labels.csv         |   2 +
.../fixtures/clean_single_rally/provenance.json    |  23 +
.../fixtures/clean_single_rally/trajectory.csv     | 121 +++
eval_baselines/fixtures/manifest.json              |  62 ++
.../multi_rally_with_deadtime/expected.json        |  54 ++
.../fixtures/multi_rally_with_deadtime/labels.csv  |   3 +
.../multi_rally_with_deadtime/provenance.json      |  23 +
.../multi_rally_with_deadtime/trajectory.csv       | 331 ++++++++
.../fixtures/occlusion_break/expected.json         |  54 ++
eval_baselines/fixtures/occlusion_break/labels.csv |   3 +
.../fixtures/occlusion_break/provenance.json       |  23 +
.../fixtures/occlusion_break/trajectory.csv        | 331 ++++++++
eval_baselines/nightly_baseline.json               | 569 ++++++++++++++
mypy.ini                                           |  13 +
pyproject.toml                                     |  17 +
requirements.txt                                   |   1 +
scripts/doctor.py                                  |  31 +-
scripts/nightly_regression.ps1                     |  83 ++
scripts/nightly_regression.py                      | 633 ++++++++++++++
scripts/register_nightly_task.ps1                  |  70 ++
scripts/run_phase3_eval.py                         |  11 +-
scripts/run_process_and_stitch.py                  |  23 +-
tests/test_ablation.py                             |  47 +-
tests/test_api_endpoints.py                        | 396 +++++++++-
tests/test_app_home.py                             | 187 +++++
tests/test_async_jobs.py                           |   8 +-
tests/test_audio_features.py                       |  26 +
tests/test_billing.py                              | 192 +++++
tests/test_bind_and_host.py                        |  75 ++
tests/test_compute_backend.py                      | 289 +++++++
tests/test_config.py                               |  28 +-
tests/test_config_deployment.py                    | 371 +++++++++
```

```
 tests/test_config_wasb.py                          |  16 +-
 tests/test_court_gate.py                           | 103 +++
 tests/test_database.py                             | 170 ++++-
 tests/test_device_context.py                       |  47 ++
 tests/test_doctor.py                               |  67 ++
 tests/test_edge_auth.py                            | 231 ++++++
 tests/test_experiment.py                           |  72 ++
 tests/test_fusion_compare.py                       |  25 +
 tests/test_fusion_hybrid.py                        |  10 +
 tests/test_generate_highlights.py                  | 174 ++++-
 tests/test_golden_fixtures.py                      | 138 ++++
 tests/test_golden_fixtures_split.py                |  87 +++
 tests/test_golden_real_fixtures.py                 | 504 ++++++++++++
 tests/test_input_source.py                         | 152 ++++
 tests/test_job_worker_loop.py                      |   8 +-
 tests/test_job_worker_module_seam.py               |  97 +++
 tests/test_motion_segmenter.py                     | 222 ++++++
 tests/test_native_wasb_runner.py                   | 383 ++++++++++
 tests/test_nightly_regression.py                   | 343 +++++++++
 tests/test_output_base.py                          |  90 +++
 tests/test_profile_parity.py                       | 140 ++++
 tests/test_provider_gemini.py                      |  21 +
 tests/test_rally_seg_eval.py                       | 155 +++-
 tests/test_rally_seg_eval_report.py                | 306 ++++++++
 tests/test_redact.py                               |  78 ++
 tests/test_results.py                              |  50 ++
 tests/test_served_gate_a_regression.py             |  96 ++-
 tests/test_server_entrypoint.py                    | 177 +++++
 tests/test_single_instance.py                      |  54 ++
 tests/test_startup_checks.py                       | 369 +++++++++
 tests/test_storage.py                              | 328 ++++++++
 tests/test_stub_runner.py                          | 129 ++++
 tests/test_tolerance_metrics.py                    | 188 +++++
 tests/test_tracknet_runner.py                      |  96 +++
 tests/test_training_gen0.py                        |  56 ++
 tests/test_upload_download.py                      |   3 +-
 tests/test_wasb_cache.py                           |  20 +
 tests/test_windowing.py                            |  37 +-
 tests/test_worker.py                               | 309 ++++++++
 tests/test_workspace.py                            | 122 +++
 tests/test_yolo_hybrid.py                          |  41 +
 tools/generate_highlights.py                       | 118 ++-
 training/gen0/harness.py                           |  50 +-
 211 files changed, 23489 insertions(+), 1395 deletions(-)
```

6. assistant (2026-06-21T17:21:14.115Z)

The diff vs master is huge (this branch is far ahead). Let me focus on the specific P1a changes by looking at the uncommitted working-tree diff, which is what this PR actually adds.

7. user (2026-06-21T17:21:15.519Z)

```
 backend/infrastructure/database.py |  18 ++++++
 backend/main.py                    | 121 ++++++++++++++++++++++++++++++++++++
 tests/test_api_endpoints.py        |  74 ++++++++++++++++++++++
 3 files changed, 213 insertions(+)
```

8. assistant (2026-06-21T17:21:21.769Z)

The actual P1a change is the uncommitted working-tree diff. Let me read it in full.

9. user (2026-06-21T17:21:22.804Z)

```
diff --git a/backend/main.py b/backend/main.py
```

```
index 2c1e344..3050c2e 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -2,7 +2,9 @@ import os
 import sys
 import argparse
 import logging
+import shutil
 import uuid
+from importlib.metadata import PackageNotFoundError, version as _pkg_version
 from urllib.parse import quote
 import uvicorn
 from dotenv import load_dotenv
@@ -44,6 +46,7 @@ from backend.config import (  # new typed config
     StitchingConfig,
     enforce_startup_checks,
     StartupCheckError,
+    probe_cuda_available,  # cheap env-based GPU probe (no torch import) for /api/capabilities
 )
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
@@ -228,6 +231,124 @@ def health():
         "default_segmenter": getattr(indexing, "default_segmenter", None),
     }

+
+# P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
+# endpoint's request/response shape changes incompatibly (purely additive endpoints need no
bump).
+API_VERSION = 1
+
+
+def _engine_version() -> str:
+    """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
+    try:
+        return _pkg_version("badminton-highlight-indexer")
+    except PackageNotFoundError:
+        return "unknown"
+
+
+@app.get("/api/version")
+def api_version():
+    """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md P1)."""
+    return {"engine_version": _engine_version(), "api_version": API_VERSION}
+
+
+@app.get("/api/capabilities")
+def capabilities():
+    """Honest probe of what THIS box can do ? the first-run wizard renders the compute step
from
+    it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT, never
its
+    value. Best-effort + never raises (the wizard degrades gracefully)."""
+    indexing = getattr(global_config, "indexing", None)
+    deployment = getattr(global_config, "deployment", None)
+    ffmpeg = shutil.which("ffmpeg")
+    ffprobe = shutil.which("ffprobe")
+    # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
+    wasb = getattr(indexing, "wasb", None)
+    weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
+    try:
+        cuda = probe_cuda_available()  # cheap, env-based (no torch import)
+    except Exception:
```

```
+        cuda = False
+    return {
+        "api_version": API_VERSION,
+        "engine_version": _engine_version(),
+        "segmenters": sorted(segmenter_registry.list_available().keys()),
+        "default_segmenter": getattr(indexing, "default_segmenter", None),
+        "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
+                   "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
+        "gpu": {"cuda_available": bool(cuda)},
+        "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
+        "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
+        "deployment": {
+            "profile": getattr(deployment, "profile", "") or "",
+            "compute_target": getattr(deployment, "compute_target", "") or "",
+        },
+    }
+
+
+@app.get("/api/videos")
+def list_videos(limit: Optional[int] = None):
+    """List indexed videos (most recent first) so the console survives a reload ? previously
only a
+    single-row get existed (PACKAGING_PLAN.md P1)."""
+    return {"videos": db_instance.list_videos(limit=limit)}
+
+
+def _reels_dir() -> str:
+    return getattr(getattr(global_config, "output", None), "output_dir", "output") or "output"
+
+
+_REEL_EXTS = (".mp4", ".mov")
+
+
+@app.get("/api/reels")
+def list_reels():
+    """List compiled highlight reels present in the output dir, newest first, with a download
URL.
+
+    NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
+    does NOT track a video_id?reel association today (a per-video listing would be a phantom),
so
+    this is a global list. (Per-video reel tracking would need a DB record + compile change ? a
+    later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
+    base = _reels_dir()
+    reels = []
+    try:
+        entries = os.scandir(base)
+    except OSError:
+        return {"reels": []}
+    with entries:
+        for e in entries:
+            if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
+                try:
+                    st = e.stat()
+                except OSError:
+                    continue
+                reels.append({
+                    "filename": e.name,
+                    "size_bytes": st.st_size,
+                    "modified_at": st.st_mtime,
+                    "download_url": f"/api/reels/{quote(e.name)}",
+                })
+    reels.sort(key=lambda r: r["modified_at"], reverse=True)
```

```
+    return {"reels": reels}
+
+
+@app.get("/api/reels/{filename}")
+def download_reel(filename: str):
+    """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
+    Mirrors /api/highlights' safety (bare-name + containment checks)."""
+    try:
+        name = safe_output_filename(filename)
+    except ValueError as e:
+        raise HTTPException(status_code=400, detail=str(e)) from e
+    base = _reels_dir()
+    path = os.path.join(base, name)
+    try:
+        path = resolve_under_root(path, base)
+    except ValueError as e:
+        raise HTTPException(status_code=400, detail=str(e)) from e
+    if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
+        raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
+    media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
+    return FileResponse(path, media_type=media, filename=name)
+
+
 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.
```

10. assistant (2026-06-21T17:21:28.750Z)

Now let me read the supporting helpers referenced: `safe_output_filename`, `resolve_under_root`, the existing `/api/config`, `/api/highlights`, and `/api/compile` handlers, and the existing `/api/videos` (get_video) for comparison.

11. user (2026-06-21T17:21:30.327Z)

```
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 4e0ac4a..aa9c415 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -476,6 +476,24 @@ class Database:
                 return d
         return None

+    def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
+        """All indexed videos, most-recently-added first (the videos table has no timestamp, so
+        order by rowid). Mirrors get_video's row shape (validation_results parsed). Powers
+        GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
+        sql = "SELECT * FROM videos ORDER BY rowid DESC"
+        params: tuple = ()
+        if limit is not None:
+            sql += " LIMIT ?"
+            params = (int(limit),)
+        with self._connect() as conn:
+            rows = conn.cursor().execute(sql, params).fetchall()
+        out: List[Dict[str, Any]] = []
+        for row in rows:
+            d = dict(row)
+            d["validation_results"] = json.loads(d["validation_results"] or "[]")
+            out.append(d)
+        return out
+
     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
         """
         Dynamically queries play segments based on filters:
```

12. user (2026-06-21T17:21:33.819Z)

```
diff --git a/tests/test_api_endpoints.py b/tests/test_api_endpoints.py
index f1c6d93..1da9733 100644
--- a/tests/test_api_endpoints.py
+++ b/tests/test_api_endpoints.py
@@ -78,6 +78,80 @@ def test_api_config_redacts_secret_shaped_fields(env):
     assert r.json()["sport"] == "badminton"  # non-secret fields still served


+# --- P1: net-new endpoints the operator console / first-run wizard depend on ---
+
+def test_api_version(env):
+    client = env[0]
+    r = client.get("/api/version")
+    assert r.status_code == 200
+    body = r.json()
+    assert body["api_version"] == m.API_VERSION
+    assert "engine_version" in body
+
+
+def test_api_capabilities_shape_and_no_secret_leak(env, monkeypatch):
+    client = env[0]
+    monkeypatch.setenv("GEMINI_API_KEY", "AIza-should-not-leak")
+    r = client.get("/api/capabilities")
+    assert r.status_code == 200
+    body = r.json()
+    assert isinstance(body["segmenters"], list) and "motion" in body["segmenters"]
+    assert "default_segmenter" in body
+    assert set(body["ffmpeg"]) >= {"ffmpeg", "ffprobe"}
+    assert isinstance(body["gpu"]["cuda_available"], bool)
+    assert body["gemini_key_present"] is True            # presence reported...
+    assert "AIza-should-not-leak" not in r.text          # ...but never the value
+    assert "profile" in body["deployment"]
+
+
+def test_api_capabilities_gemini_absent(env, monkeypatch):
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    assert env[0].get("/api/capabilities").json()["gemini_key_present"] is False
+
+
+def test_api_videos_lists_all_newest_first(env):
+    client, db, tmp_path, mp = env
+    db.add_video("vid_a", "/a.mp4", "processed", [])
+    db.add_video("vid_b", "/b.mp4", "processed", [])
+    body = client.get("/api/videos").json()
+    ids = [v["id"] for v in body["videos"]]
+    assert ids == ["vid_b", "vid_a"]  # most-recently-added first
+
+
+def test_api_videos_empty(env):
+    assert env[0].get("/api/videos").json()["videos"] == []
+
+
+def test_api_reels_lists_and_downloads(env):
+    client, db, tmp_path, mp = env
+    reels_dir = tmp_path / "out"
+    reels_dir.mkdir()
+    mp.setattr(m.global_config.output, "output_dir", str(reels_dir))
+    (reels_dir / "reel_a.mp4").write_bytes(b"\x00\x00mp4")
+    (reels_dir / "reel_b.mov").write_bytes(b"\x00\x00mov")
+    (reels_dir / "notes.txt").write_text("not a reel", encoding="utf-8")
+    (reels_dir / "vid_subdir").mkdir()  # per-video scratch subdir ? must be skipped
+
+    body = client.get("/api/reels").json()
```

```
+    names = {r["filename"] for r in body["reels"]}
+    assert names == {"reel_a.mp4", "reel_b.mov"}  # only top-level video files
+    a = next(r for r in body["reels"] if r["filename"] == "reel_a.mp4")
+    assert a["download_url"] == "/api/reels/reel_a.mp4" and a["size_bytes"] == 5
+
+    # the advertised download URL works...
+    dl = client.get(a["download_url"])
+    assert dl.status_code == 200 and dl.headers["content-type"].startswith("video/")
+    # ...and a missing / non-reel name 404s
+    assert client.get("/api/reels/missing.mp4").status_code == 404
+    assert client.get("/api/reels/notes.txt").status_code == 404
+
+
+def test_api_reels_empty_when_no_output_dir(env):
+    client, db, tmp_path, mp = env
+    mp.setattr(m.global_config.output, "output_dir", str(tmp_path / "does_not_exist"))
+    assert env[0].get("/api/reels").json()["reels"] == []
+
+
 def test_process_success_with_mocked_segmenter(env):
     """#16: submit returns 202 + job_id; the (inline) job carries the old response."""
     client, db, tmp_path, mp = env
```

13. assistant (2026-06-21T17:21:36.836Z)

Now let me read the supporting helpers in main.py: `safe_output_filename`, `resolve_under_root`, the existing `/api/config`, `/api/highlights`, `/api/compile` handlers, and the imports.

14. user (2026-06-21T17:21:37.826Z)

Found 2 files
C:\Users\avidu\Projects\bhi-wt-p1a\backend\utils\paths.py
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py

15. user (2026-06-21T17:21:40.283Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Any, Optional
4
5
6       def output_base(config: Any) -> str:
7           """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8           (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
9           chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10          a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12          This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13          ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy
dict-shim
14          (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
15          try:
16              out = config.get("output", {}).get("output_dir")
17          except Exception:
18              out = None
19          return out or "output"
20
21
22      def safe_output_filename(name: str) -> str:
```

```
23          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
24
25          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
26          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
27          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
28          """
29          if not name or not name.strip():
30              raise ValueError("output_filename must not be empty")
31          if "\x00" in name:
32              raise ValueError("output_filename must not contain a null byte")
33          if "/" in name or "\\" in name:
34              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
35          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
36              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
37          return name
38
39
40      def resolve_under_root(path: str, root: str) -> str:
41          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
42          rp = os.path.realpath(path)
43          rr = os.path.realpath(root)
44          try:
45              contained = os.path.commonpath([rp, rr]) == rr
46          except ValueError:  # different drives on Windows
47              contained = False
48          if not contained:
49              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
50          return rp
51
52
53      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
54          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
57          so the local 'point it at any of my videos' workflow is preserved. Setting an
58          allowed_root (a future hosted/tenant deployment) enforces containment.
59          """
60          if "\x00" in path:
61              raise ValueError("video_path must not contain a null byte")
62          if allowed_root:
63              resolve_under_root(path, allowed_root)
64
```

16. assistant (2026-06-21T17:21:45.206Z)

Now let me look at the existing `/api/config`, `/api/highlights`, and `/api/compile` handlers in main.py to verify consistency and the compile-flat-storage claim.

17. user (2026-06-21T17:21:46.617Z)

```
34:from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
40:from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
41:from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
211:@app.get("/api/config")
212:def get_config():
216:    # by tests/test_redact.py + the /api/config redaction test).
305:    NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
335:    Mirrors /api/highlights' safety (bare-name + containment checks)."""
```

```
337:        name = safe_output_filename(filename)
343:        path = resolve_under_root(path, base)
349:    return FileResponse(path, media_type=media, filename=name)
582:@app.get("/api/highlights/{video_id}/{filename}")
583:def download_highlight(video_id: str, filename: str):
584:    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
588:        name = safe_output_filename(filename)
596:        path = resolve_under_root(path, output_dir)
601:                            detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
603:    return FileResponse(path, media_type=media, filename=name)
617:@app.post("/api/compile")
618:def compile_highlights(req: CompileRequest):
633:        out_name = safe_output_filename(req.output_filename)
677:        download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
851:    # /api/compile ? reads the same value instead of a write-only runtime hack.
```

18. user (2026-06-21T17:21:50.252Z)

```
205             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
206         else:
207             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
208
209
210     # --- API Endpoints ---
211     @app.get("/api/config")
212     def get_config():
213         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
214         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
215         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
216         # by tests/test_redact.py + the /api/config redaction test).
217         if global_config is None:
218             return {}
219         return redact_secrets(global_config.model_dump(mode="json"))
220
221     @app.get("/api/health")
222     def health():
223         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
224         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
225         confirm the engine is reachable and which detector is wired. Never raises."""
226         sport = getattr(global_config, "sport", None)
227         indexing = getattr(global_config, "indexing", None)
228         return {
229             "status": "ok",
230             "sport": sport,
231             "default_segmenter": getattr(indexing, "default_segmenter", None),
232         }
233
234
```

19. assistant (2026-06-21T17:21:53.087Z)

Now let me read the `/api/highlights` and `/api/compile` handlers to confirm the flat-storage claim and compare safety.

20. user (2026-06-21T17:21:53.703Z)

```
582     @app.get("/api/highlights/{video_id}/{filename}")
583     def download_highlight(video_id: str, filename: str):
584         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
```

```
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
```

```
646             try:
647                 if sc is not None and int(sc) < stitching.min_shot_count:
648                     continue
649             except (TypeError, ValueError):
650                 pass  # unparseable count -> treat as unknown, keep
651             kept.append(s)
652         if len(kept) < len(matched_segments):
653             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
654                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
655         if not kept:
656             raise HTTPException(status_code=400,
657                                 detail=f"All {len(matched_segments)} matching segments fall
below "
658
f"stitching.min_shot_count={stitching.min_shot_count}.")
659
660         # Extract start/end time pairs
661         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
662
663         # Run compiler with the configured stitching paddings + optional branding watermark
664         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
665         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
666                                  end_padding=stitching.end_padding,
watermark=stitching.watermark)
667         try:
668             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
669             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
670             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
671             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
672             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
673             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
674             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
675             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
676             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
677             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
678             return {"status": "success", "filepath": out_path, "download_url": download_url}
679         except Exception as e:
680             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
681
```

21. assistant (2026-06-21T17:21:58.329Z)

Confirmed: `/api/compile` writes via `VideoCompiler(output_dir, ...)` to `output_dir` flat under `out_name`. Now let me verify the `VideoCompiler.compile_highlights` writes flat (not into a per-video subdir), to confirm the "flat" claim and the "scratch subdirs" claim.

22. user (2026-06-21T17:21:59.982Z)

```
17:    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
19:        self.output_dir = output_dir
25:        os.makedirs(output_dir, exist_ok=True)
164:            tf_path = os.path.join(tmp_dir, "watermark.txt")
233:        clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
```

```
303:    def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
324:        # Defense-in-depth: never let the output name escape output_dir (the API also
326:        output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
339:        # Create a temp directory within output_dir
340:        with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
375:            list_file_path = os.path.join(tmp_dir, "clips.txt")
```

23. assistant (2026-06-21T17:22:04.063Z)

Confirmed: `compile_highlights` writes flat to `os.path.join(self.output_dir, os.path.basename(output_filename))` ? so reels are flat in `output_dir`, and `tempfile.TemporaryDirectory(dir=self.output_dir)` creates transient subdirs (cleaned up). The `/api/reels` listing skips subdirs (`e.is_file()`), so it won't pick up scratch dirs or the DB. Let me verify where the SQLite DB and `.lock` files live relative to `output_dir`.

24. user (2026-06-21T17:22:05.366Z)

```
97:def probe_cuda_available() -> bool:
98-    """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99-
100-    Exposed for the per-profile startup checks (``backend/config/startup.py``) so a caller
101-    that *wants* the GPU heads-up can pass the result in without the startup module
importing
102-    torch itself. Best-effort by design: if torch lives only in a detector subprocess env
103-    (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104-    treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105-    error that could false-fail."""
106-    return _cuda_available()
107-
108-
109-def suggest_profile(
110-    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111-) -> str:
112-    """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113-
114-    Signals, in priority order:
115-
116-      - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
117-      - ``CI`` truthy                        ? ``"cpu-mock"``
118-      - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119-      - otherwise                            ? ``""`` (no strong suggestion)
120-
121-    ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122-    imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
```

25. assistant (2026-06-21T17:22:08.765Z)

The docstring says "lazy torch import" ? that contradicts the inline comment in main.py claiming "no torch import". Let me read the actual `_cuda_available` implementation to determine if it actually imports torch.

26. user (2026-06-21T17:22:09.771Z)

```
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py:84:def _cuda_available() -> bool:
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-85-    """Best-effort CUDA probe.
torch is heavy + optional, so it is imported lazily here and
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-86-    ONLY when a caller opts into
``probe_cuda``; any import/runtime failure means 'no GPU'."""
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-87-    try:
```

```
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-88-        # Intentionally lazy +
local: keeps the config-load path torch-free (torch is heavy
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-89-        # and optional). Only
reached when a caller opts into probe_cuda.
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py:90:        import torch
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-91-
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py:92:        return
bool(torch.cuda.is_available())
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-93-    except Exception:
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-94-        return False
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-95-
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-96-
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-97-def probe_cuda_available() ->
bool:
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-98-    """Public best-effort CUDA
probe (lazy torch import; ``False`` on any failure).
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-99-
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-100-    Exposed for the per-profile
startup checks (``backend/config/startup.py``) so a caller
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-101-    that *wants* the GPU heads-
up can pass the result in without the startup module importing
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-102-    torch itself. Best-effort
by design: if torch lives only in a detector subprocess env
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-103-    (native WASB), this returns
``False`` in the main process ? which is why the startup checks
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-104-    treat a GPU mismatch as a
WARNING (the runner healthcheck is the authority), never a hard
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-105-    error that could false-
fail."""
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-106-    return _cuda_available()
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-107-
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-108-
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-109-def suggest_profile(
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-110-    env: Optional[Mapping[str,
str]] = None, *, probe_cuda: bool = False
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-111-) -> str:
C:\Users\avidu\Projects\bhi-wt-p1a\backend\config\presets.py-112-    """SUGGEST (never auto-
apply ? D15) a deployment profile from the environment.
```

27. assistant (2026-06-21T17:22:14.910Z)

Important finding: `probe_cuda_available()` ? `_cuda_available()` actually does `import torch; torch.cuda.is_available()`. The inline comment in main.py at the import (line 49) and at line 261 says "cheap, env-based (no torch import)" ? this is **factually wrong**. It imports torch (heavy) and calls `torch.cuda.is_available()` (which initializes CUDA driver). The Q5 premise ("genuinely cheap, no torch import") is violated. Let me confirm there's no separate env-based variant being referenced.

Now let me check the `env` fixture in the test file and confirm `get_video`'s row shape to verify Q4 (does `list_videos` leak more than `get_video`).

28. user (2026-06-21T17:22:15.645Z)

```
1       """TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
2       that broke `import backend.main` shipped green because nothing imported it)."""
3       import os
4       import subprocess
5       import sys
6       from unittest.mock import patch, MagicMock
7
8       import pytest
9
10      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
11      from fastapi.testclient import TestClient
12
```

```
    import backend.main as m
    from backend.infrastructure.database import Database
    from backend.config.models import AppConfig
    from backend.pipeline.validators.base import ValidationResult


    class _InlineExecutor:
        """submit() runs the fn synchronously ? async jobs complete before the response
        is inspected (full lifecycle coverage lives in test_async_jobs.py)."""

        def submit(self, fn, *args, **kwargs):
            fn(*args, **kwargs)


    @pytest.fixture
    def env(tmp_path, monkeypatch):
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        client = TestClient(m.app)
        return client, db, tmp_path, monkeypatch


    def _video(tmp_path, name="m.mp4"):
        p = tmp_path / name
        p.write_bytes(b"not-a-real-video-but-hashable")
        return p


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    def _fail():
        return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]


    class _FakeSeg:
        def __init__(self, db, cfg):
            self.db = db

        def process_video(self, video_id, video_path, *a, **k):
            sid = self.db.add_play_segment(video_id, 0.0, 5.0)
            return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]


    def test_api_config_ok(env):
        client = env[0]
        r = client.get("/api/config")
        assert r.status_code == 200
        assert r.json()["sport"] == "badminton"


    def test_api_config_redacts_secret_shaped_fields(env):
        """P0b: the unauthenticated /api/config must never serialize a credential-shaped
value.
        AppConfig stores no secrets today, so inject one via its extra='allow' tolerance and
assert
        it is masked while non-secret fields are served unchanged."""
        client, db, tmp_path, mp = env
        mp.setattr(m, "global_config", AppConfig(gemini_api_key="SECRET-LEAK-123"))
```

```
74          r = client.get("/api/config")
75          assert r.status_code == 200
76          assert "SECRET-LEAK-123" not in r.text  # the secret never crosses the wire
77          assert r.json()["gemini_api_key"] == "***REDACTED***"
78          assert r.json()["sport"] == "badminton"  # non-secret fields still served
```

29. assistant (2026-06-21T17:22:18.350Z)

Now let me look at `get_video` and the videos table schema to confirm Q4 ? what columns `list_videos` returns and whether it leaks anything beyond `get_video`.

30. user (2026-06-21T17:22:19.336Z)

```
470:    def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
```

31. user (2026-06-21T17:22:22.586Z)

```
470         def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
471             with self._connect() as conn:
472                 row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
473                 if row:
474                     d = dict(row)
475                     d["validation_results"] = json.loads(d["validation_results"] or "[]")
476                     return d
477             return None
478
479         def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
480             """All indexed videos, most-recently-added first (the videos table has no
timestamp, so
481             order by rowid). Mirrors get_video's row shape (validation_results parsed).
Powers
482             GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
483             sql = "SELECT * FROM videos ORDER BY rowid DESC"
484             params: tuple = ()
485             if limit is not None:
486                 sql += " LIMIT ?"
487                 params = (int(limit),)
488             with self._connect() as conn:
489                 rows = conn.cursor().execute(sql, params).fetchall()
490             out: List[Dict[str, Any]] = []
491             for row in rows:
492                 d = dict(row)
493                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
494                 out.append(d)
```

32. assistant (2026-06-21T17:22:28.571Z)

`list_videos` uses `SELECT *` exactly like `get_video` and parses `validation_results` identically ? so it leaks nothing beyond the existing per-row endpoint. The `limit` is parameterized via `?` placeholder with `int(limit)` coercion ? no injection. Let me check the `videos` table schema to confirm `filepath` is already exposed (Q4 mentions /api/videos filepaths as accepted disclosure) and confirm `rowid` ordering is sane (no explicit `WITHOUT ROWID` or custom PK aliasing `id` to rowid).

33. user (2026-06-21T17:22:29.988Z)

```
118:            CREATE TABLE IF NOT EXISTS videos (
119-                id TEXT PRIMARY KEY,
120-                filepath TEXT NOT NULL,
121-                processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
122-                status TEXT NOT NULL,
123-                validation_results TEXT
124-            )
```

```
125-        """)
126-
127-        # Play Segments Table (Extensible metadata JSON)
128-        cursor.execute("""
129-            CREATE TABLE IF NOT EXISTS play_segments (
130-                id INTEGER PRIMARY KEY AUTOINCREMENT,
```

34. assistant (2026-06-21T17:22:39.005Z)

Two findings here regarding the `list_videos` docstring claim "the videos table has no timestamp":

1. The table **does** have a `processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` column. So the docstring statement "the videos table has no timestamp, so order by rowid" is **factually inaccurate**. However, `id TEXT PRIMARY KEY` (not `INTEGER PRIMARY KEY`), so `rowid` is a genuine separate monotonic insertion counter ? ordering by `rowid DESC` for "most-recently-added" is correct and arguably more reliable than `processed_at` (which has 1-second granularity and could tie). So the ordering behavior is fine; only the docstring rationale is wrong.

Let me verify the test for newest-first ordering relies on rowid (two videos added in the same second would tie on `processed_at`), confirming the ordering choice is actually sound, and double-check `processed_at` is the only timestamp. Now let me look at whether `id TEXT PRIMARY KEY` means an explicit `rowid` exists (it does, since the table is not `WITHOUT ROWID`). Good.

Now let me verify Q2 path-safety edge: the FastAPI `{filename}` route ? does it capture path separators? With a single `{filename}` (not `{filename:path}`), FastAPI/Starlette will not match across `/`. Let me also double check that the redact applies and check the `quote` import and the `Optional` import for `list_videos(limit: Optional[int])`.

35. user (2026-06-21T17:22:39.786Z)

```
28      from fastapi.middleware.cors import CORSMiddleware
29      from starlette.middleware.trustedhost import TrustedHostMiddleware
30      from pydantic import BaseModel
31      from typing import List, Dict, Any, Optional
32
33      from fastapi.staticfiles import StaticFiles
34      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
35      from backend.pipeline.validators.base import registry
36      from backend.infrastructure.database import Database
37      from backend.pipeline.segmenters.base import segmenter_registry
38      from backend.pipeline.stitcher.compiler import VideoCompiler
39      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
40      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
41      from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
42      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
43      from backend.config import (  # new typed config
44          load_config as load_app_config,
45          AppConfig,
46          StitchingConfig,
47          enforce_startup_checks,
```

36. assistant (2026-06-21T17:22:42.524Z)

`Optional`, `quote`, `FileResponse`, `HTTPException` are all imported. Now let me confirm `HTTPException` is imported (used in `download_reel`).

37. user (2026-06-21T17:22:43.742Z)

```
27:from fastapi import FastAPI, HTTPException, Request
339:        raise HTTPException(status_code=400, detail=str(e)) from e
345:        raise HTTPException(status_code=400, detail=str(e)) from e
```

[Showing results with pagination = limit: 3]

38. assistant (2026-06-21T17:22:53.805Z)

All imports present. Now let me verify the Q2 traversal concern more carefully. The route is `/api/reels/{filename}` (single-segment). Starlette's default `{filename}` converter is `str` which does NOT match `/`. So a request like `/api/reels/../foo` ? the `..` would be normalized by the HTTP client/router before reaching the path param. And even if it reached, `safe_output_filename` rejects `/`, `\`, `..`, absolute. The `resolve_under_root` is defense-in-depth. The `.mp4`/`.mov` ext gate is applied AFTER `os.path.isfile` ? let me re-examine: `if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS)`.

This is correct ? both conditions must be satisfied to serve. There's a subtle redundancy: `safe_output_filename` already guarantees `name` is a bare filename, so `resolve_under_root` can only fail on symlink escape. That's the genuine value-add (TOCTOU/symlink): if `output_dir/reel_a.mp4` is a symlink pointing outside, `resolve_under_root` (realpath) catches it. Good ? symlink-safe.

Let me check Q3 about the `.lock` file location (P0e single-instance lock) relative to output_dir, to confirm it isn't a top-level `.mp4`/`.mov` (it won't be ? it's `.lock`), but more importantly whether the SQLite DB could ever live in output_dir with a video extension. The DB is `t.db` / configured db path ? not `.mp4`/`.mov`, so the extension gate excludes it regardless. Let me confirm the lock file naming.

39. user (2026-06-21T17:22:54.858Z)

```
1       """Single-instance file lock (PACKAGING_PLAN.md P0e).
2
3       The serving model runs an IN-PROCESS job worker, and ``fail_stale_jobs()`` sweeps any
job left
4       ``queued``/``running`` by a "previous" process to ``failed`` on startup. If a SECOND
process opens the
5       same SQLite DB (a second ``indexer --server``, or a CLI run while a server is up), its
sweep would
6       mark the FIRST process's in-flight jobs failed ? corrupting live state. This module
guards the DB
7       with an exclusive OS lock so the second instance refuses to start instead.
8
9       OS-managed advisory lock (``fcntl.flock`` on POSIX, ``msvcrt.locking`` on Windows): held
for the
10      process lifetime via an open file handle and **released automatically by the OS on exit
OR crash**,
11      so there is no stale-lock problem (unlike a bare PID file). The lock is keyed to the
lock-file path,
12      which callers co-locate with the DB ? so two instances on DIFFERENT ``--output-dir``
(different DBs)
13      coexist fine.
14      """
15
16      from __future__ import annotations
17
18      import os
19      import sys
20      from typing import IO, Optional
21
22      # NB: branch on ``sys.platform == "win32"`` (NOT os.name) so mypy's platform narrowing
analyses only
23      # the matching branch ? the POSIX-only ``fcntl`` and Windows-only ``msvcrt`` never
cross-error.
24
25
26      def acquire_lock(lock_path: str) -> Optional[IO[str]]:
27          """Try to take an exclusive, NON-blocking lock on ``lock_path``.
28
29          Returns the open file handle on success (KEEP IT ALIVE for the process lifetime ?
```

```
letting it be
30          garbage-collected/closed releases the lock), or ``None`` if another live process
holds it.
31          """
32          parent = os.path.dirname(os.path.abspath(lock_path))
33          if parent:
34              os.makedirs(parent, exist_ok=True)
35          fh = open(lock_path, "a+")
36          try:
37              if sys.platform == "win32":
38                  import msvcrt
39                  fh.seek(0)
40                  msvcrt.locking(fh.fileno(), msvcrt.LK_NBLCK, 1)  # lock 1 byte at offset 0,
non-blocking
```

40. assistant (2026-06-21T17:23:00.549Z)

The lock file is co-located with the DB (not necessarily output_dir), and it's a `.lock` extension regardless ? excluded by the `.mp4`/`.mov` gate. The DB co-location is fine. Even if the DB and lock were inside output_dir, the extension gate excludes them.

Now let me double-check the Q5 robustness path: when `global_config is None`. In `capabilities()`, `getattr(global_config, "indexing", None)` ? `getattr(None, "indexing", None)` ? `None`. Then `getattr(wasb, "weights_path", "")` with `wasb=getattr(None,"wasb",None)=None` ? `""`. All `getattr` chains are None-safe. `segmenter_registry.list_available()` ? let me verify it doesn't raise when empty (returns a dict). And confirm the `probe_cuda_available` is wrapped in try/except (it is). Let me verify `list_available` returns a dict.

41. user (2026-06-21T17:23:01.869Z)

```
51:    def list_available(self) -> Dict[str, str]:
52-        """Returns a dict of registered segmenter names and their descriptions."""
53-        res = {}
54-        for name, cls in self._segmenters.items():
55-            try:
56-                temp_inst = cls(None, {})
57-                desc = temp_inst.description
```

42. user (2026-06-21T17:23:05.105Z)

```
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

43. assistant (2026-06-21T17:23:16.403Z)

`list_available()` is robust (per-segmenter try/except, returns dict; empty registry returns `{}`). So `capabilities()` is genuinely None-safe and won't raise from registry or config ? except the **one remaining risk**: `os.path.exists(weights_path)` is called unconditionally inside the return dict. With `weights_path=""`, `os.path.exists("")` returns `False` safely (no raise). `shutil.which` doesn't raise. So `capabilities()` does hold its "never raises" contract ? **but** the torch import inside `probe_cuda_available` is the heavy part, and it IS wrapped in try/except, so it won't raise; it's just not "cheap".

Now let me verify the Q1 secret-leak concern fully ? that `wasb_weights_present` is a bool (it is: `bool(weights_path) and os.path.exists(...)` ? bool), and that no full secret path (the weights path or gemini key) is serialized. The weights_path is NOT serialized (only the bool). The gemini key is NOT serialized (only `bool(os.environ.get(...))`). Confirmed.

But Q1 also asks about `ffmpeg_path`/`ffprobe_path` (full filesystem paths) disclosure. These ARE serialized (`"ffmpeg_path": ffmpeg`). On an unauthenticated localhost endpoint this discloses the install location (e.g. `C:\Users\<username>\...\ffmpeg.exe`), which leaks the OS username and install layout. Consistency check: `/api/config` is now redacted (P0b) but still serializes non-secret config including `output.output_dir` (a path), and `/api/videos` serializes `filepath` (full source video paths). So path disclosure is already the established posture for these localhost endpoints ? `ffmpeg_path` is consistent with that, not a new class of leak. I'll flag it as a minor/nit for awareness, not a blocker.

Now let me run the targeted P1 tests to confirm green, and confirm the torch-import behavior empirically.

44. user (2026-06-21T17:23:27.788Z)

```
.........                                                                [100%]
9 passed, 29 deselected in 7.00s
```

45. assistant (2026-06-21T17:23:30.853Z)

All 9 P1 tests pass. Let me empirically confirm whether torch is installed (and thus whether `probe_cuda_available` is actually heavy) in this env, to calibrate the severity of the "no torch import" claim.

46. user (2026-06-21T17:23:31.992Z)

```
torch installed: True
```

47. user (2026-06-21T17:23:43.082Z)

```
import time: 0.385s
first call (torch import + cuda init): 3.456s -> True
second call (torch cached): 0.000s
```

48. assistant (2026-06-21T17:23:50.777Z)

Empirically confirmed: the **first** call to `/api/capabilities` takes ~3.5s because it imports torch and initializes CUDA ? directly contradicting the inline comments at main.py:49 and :261 claiming "cheap, env-based (no torch import)". Subsequent calls are cached (0ms). This is a correctness/honesty issue in the comment, and a real first-call latency the first-run wizard will hit. Not a blocker (it's wrapped, returns correct value, caches), but the comment is materially false and worth fixing.

Let me do a final check on the broader test file to ensure nothing else in the P1 region was missed, and confirm the `_REEL_EXTS` tuple `.endswith` behavior with a tuple (valid in Python). `str.endswith(tuple)` is valid. Good.

Let me verify one more Q2 edge: the FastAPI single-segment route genuinely rejects encoded traversal. The test only checks `missing.mp4` and `notes.txt`. Let me confirm an encoded-slash / dotdot request behaves safely.

49. user (2026-06-21T17:24:02.264Z)

```
22:54:01 [INFO] httpx: HTTP Request: GET http://testserver/api/reels/..%2f..%2fbackend%2fmain.py
"HTTP/1.1 404 Not Found"
22:54:01 [INFO] httpx: HTTP Request: GET http://testserver/api/reels/%2e%2e%2f%2e%2e%2fmain.py
"HTTP/1.1 404 Not Found"
22:54:01 [INFO] httpx: HTTP Request: GET http://testserver/api/reels/foo%00.mp4 "HTTP/1.1 400
Bad Request"
22:54:01 [INFO] httpx: HTTP Request: GET http://testserver/api/reels/sub%5cevil.mp4 "HTTP/1.1
```

400 Bad Request"
404 /api/reels/..%2f..%2fbackend%2fmain.py
404 /api/reels/%2e%2e%2f%2e%2e%2fmain.py
400 /api/reels/foo%00.mp4
400 /api/reels/sub%5cevil.mp4

50. assistant (2026-06-21T17:24:35.332Z)

Path safety holds empirically:
- `..%2f..%2f...` (encoded `/`) ? Starlette decodes `%2f` to `/` inside the single-segment param, then `safe_output_filename` would reject it ? but here it 404s (the decoded `../../backend/main.py` doesn't match a `.mp4`/`.mov` and isn't a file in output_dir; the path containment + ext gate catch it). Either way, no escape.
- Null byte ? 400 (rejected by `safe_output_filename`).
- Backslash ? 400 (rejected).

Traversal is not reachable. Q2 is clean.

I have enough evidence for all six review axes. Here are my findings.

Findings

[minor] backend/main.py:49 and backend/main.py:261 ? The inline comments `# cheap env-based GPU probe (no torch import)` and `cuda = probe_cuda_available()  # cheap, env-based (no torch import)` are factually false. `probe_cuda_available()` ? `presets.py:106 _cuda_available()` ? `import torch; torch.cuda.is_available()` (presets.py:90-92). Measured: first `/api/capabilities` call takes ~3.5s (torch import + CUDA init); subsequent calls are cached (0ms). The function is correctly wrapped in try/except so it never raises and returns the right bool, but it is NOT cheap and NOT env-based on the first call. **Fix:** correct both comments to "best-effort CUDA probe (lazy torch import, ~3s first call, cached after; never raises)", or, if first-run-wizard latency matters, gate the torch probe behind a query param / cache it at startup.

[minor] backend/infrastructure/database.py:480 ? `list_videos` docstring claims "the videos table has no timestamp, so order by rowid", but the table DOES have `processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` (database.py:121). The ordering choice itself is *correct and arguably better* ? `id` is `TEXT PRIMARY KEY` so `rowid` is a true monotonic insertion counter, immune to the 1-second granularity ties `processed_at` would suffer. Only the stated rationale is wrong. **Fix:** reword to "order by rowid (monotonic insertion order; more reliable than the 1s-granularity processed_at for newest-first)".

[nit] backend/main.py:269 ? `/api/capabilities` serializes full `ffmpeg_path` / `ffprobe_path` on an unauthenticated endpoint, disclosing the install layout (and OS username on Windows paths). This is *consistent* with the established posture of the sibling localhost endpoints ? `/api/videos` returns `filepath` (full source paths) and `/api/config` returns `output.output_dir` ? so it is an acceptable disclosure under the current single-user-localhost threat model, not a regression. No secret value or full secret path is leaked: `gemini_key_present` is `bool(os.environ.get("GEMINI_API_KEY"))` and `wasb_weights_present` is `bool(weights_path) and os.path.exists(weights_path)` ? both pure bools, neither the key nor the weights path crosses the wire (verified by the test asserting `"AIza-should-not-leak" not in r.text`). **Fix (optional):** if exposure tightens later, drop `ffmpeg_path`/`ffprobe_path` to bools-only like the secret fields; fine to defer.

Axes that are clean (no issue)
- **Q1 SECRET LEAK:** Clean. Only booleans for key/weights presence; no key value, no full secret path. (`ffmpeg_path` disclosure noted above as a consistent nit.)
- **Q2 PATH SAFETY:** Clean. `safe_output_filename` rejects empty/null-byte/`/`/`\\`/absolute/`..`/non-basename; `resolve_under_root` (realpath + commonpath) is real defense-in-depth catching symlink escape; the `.mp4`/`.mov` ext gate is ANDed with `os.path.isfile`. Route is single-segment `{filename}` (not `{filename:path}`) so separators don't span. Empirically probed encoded-`/`, encoded-`..`, null-byte, backslash ? all 400/404, no escape. `os.scandir` handle is closed via `with entries:` (main.py:316). No exploitable TOCTOU

(realpath resolved, then `isfile`, then `FileResponse` opens ? worst case is a 404/serve of a since-replaced file under the same contained root, not an escape).
- **Q3 HONESTY:** Accurate. Confirmed `/api/compile` ? `VideoCompiler(output_dir).compile_highlights` writes flat to `os.path.join(self.output_dir, os.path.basename(name))` (compiler.py:326) with no `video_id` association ? the "global flat list / per-video would be a phantom" docstring is honest. `e.is_file()` correctly skips the per-video `tempfile.TemporaryDirectory(dir=output_dir)` scratch subdirs (compiler.py:340), and the SQLite DB / `.lock` file are excluded by the `.mp4`/`.mov` ext gate regardless of location. Test covers `notes.txt` + a subdir exclusion.
- **Q4 /api/videos:** Clean. SQL parameterized ? `LIMIT ?` with `int(limit)` coercion (database.py:485-487), no injection. `SELECT *` + `validation_results` JSON-parse mirrors `get_video` exactly, so it leaks nothing the existing per-row endpoint didn't already expose. `ORDER BY rowid DESC` is sound (see minor above).
- **Q5 ROBUSTNESS:** Holds the "never raises" contract. All config access is None-safe `getattr` chains (verified `global_config=None` path), `segmenter_registry.list_available()` is per-item try/excepted and returns `{}` when empty, `probe_cuda_available()` is try/excepted, `shutil.which`/`os.path.exists("")` don't raise. (The torch-import *cost*, not correctness, is the only issue ? see minor.)
- **Q6 /api/version:** Correct. `_engine_version()` catches `PackageNotFoundError` ? `"unknown"` for editable/source runs (main.py:240-244). `API_VERSION = 1` contract comment ("BUMP when an existing endpoint's shape changes incompatibly; additive endpoints need no bump") is accurate and matches the additive nature of this PR.

Verdict

Clean ? no blockers or majors; two minor doc/comment-accuracy fixes (the false "no torch import" comments at main.py:49/261 are the most worth correcting since they misrepresent a real ~3.5s first-call cost) and one optional nit. Safe to merge after the comment fix.